

WC26 Live · Pressing 90' — Documentation technique

Site officiel de prédictions et de scores temps réel pour la FIFA Coupe du Monde 2026, propulsé par la marque éditoriale **Pressing 90'**.

Live : <https://wc26.mehdijabry.dev/> **Repo :** github.com/mehdijabry/wc26-live **Stack :** Vite + React 18 + TypeScript + Tailwind CSS · Supabase · Cloudflare Pages + Workers + KV · ESPN public API · Resend SMTP

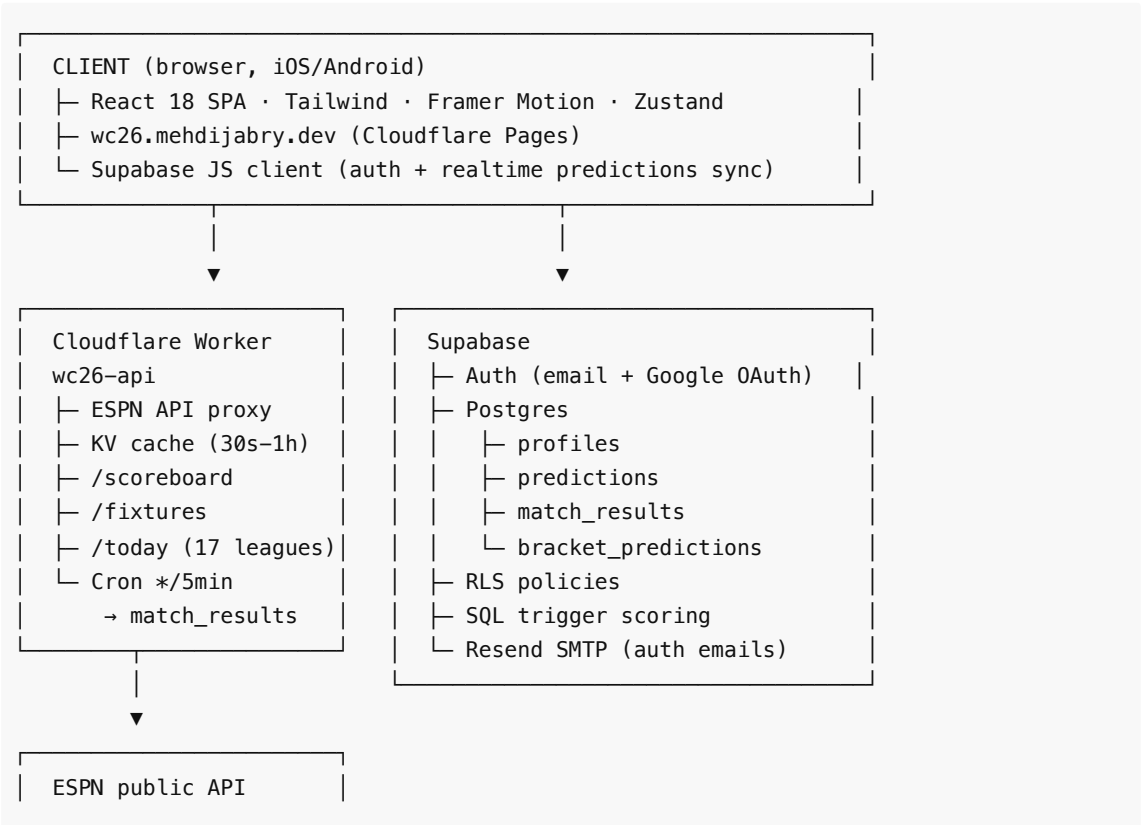
1. Vue d'ensemble

WC26 Live est une **single-page app React** déployée sur Cloudflare Pages, alimentée par :

- un **Worker Cloudflare** qui proxy et cache l'API publique d'ESPN (scores, fixtures, classements de 17 compétitions football)
- une **base Supabase Postgres** pour l'authentification, les pronostics, et le leaderboard
- un **système de scoring SQL** qui calcule la précision des pronostics en temps réel quand les résultats arrivent
- un **email transactionnel Resend** pour les magic links et les confirmations de compte
- un **bracket predictor** wizard permettant de prédire l'intégralité du tournoi et d'exporter le résultat en PNG haute résolution

Le site est **mobile-first**, **GDPR-friendly** (analytics sans cookies), et **SEO-ready** avec données structurées JSON-LD pour Google.

2. Architecture en couches



```
site.api.espn.com
(FIFA WC, UCL,
Premier League, ...)
```

3. Services tiers — coûts, rôles et raisons

Service	Rôle	Plan	Pourquoi ce choix
Cloudflare Pages	Hosting statique + CDN global	Gratuit (illimité bandwidth)	TTFB < 50ms partout, déploiements Git automatiques
Cloudflare Workers	API proxy ESPN + cron scoring	Gratuit (100k req/jour)	Edge compute, latence minimale, KV intégré
Cloudflare KV	Cache des réponses ESPN	Gratuit (1 Go)	Évite de surcharger ESPN, réponses 30s pour live, 1h pour fixtures
Cloudflare Web Analytics	Tracking visiteurs sans cookies	Gratuit	GDPR-friendly, temps réel, multi-source
Supabase	Postgres + Auth + Realtime	Gratuit (500MB DB, 50k MAU)	Open source, SQL standard, Realtime via Postgres
Resend	SMTP transactionnel	Gratuit (3000 emails/mois)	Délivrabilité premium, intégration domaine simple
Porkbun	Registrar mehdijabry.dev	~12\$/an	DNS rapide, prix imbattable, API ouverte
Google Cloud Console	OAuth client (Google login)	Gratuit	Login en 1 clic, audience massive
Google Search Console	Indexation + ranking SEO	Gratuit	Mesure des impressions et clics organiques
ESPN public API	Scores, fixtures, classements	Gratuit (non documenté officiellement)	Source unique pour 17 compétitions, format JSON propre
GitHub	Hébergement du code	Gratuit	CI/CD via Cloudflare Pages, public + sauvegarde

Coût opérationnel total : ~12 \$/an (le domaine), tout le reste est sur des tiers gratuits jusqu'à des seuils largement au-dessus de la projection de trafic d'un site de tournoi.

4. Stack frontend

Frameworks et libs principales

- **Vite 8** — bundler ultra-rapide, HMR instant en dev, build production en 5-10 secondes
- **React 18** — UI déclarative, concurrent rendering

- **TypeScript 5** — type strict, refactoring sécurisé
- **Tailwind CSS 3** — utility-first, theme custom (couleurs `ink-900` , `accent-gold` , `accent-green` , `accent-red`)
- **Framer Motion** — animations fluides (hero, modals, transitions de sections, crossfades)
- **Zustand** — state management léger avec persistance (`predictions` , `auth` , `bracket` , `flagCycle`)
- **@supabase/supabase-js** — client Supabase (auth + queries)
- **html-to-image** — conversion DOM → PNG pour l'export du bracket
- **@lottiefiles/dotlottie-react** — animation Lottie sur le hero (ballon flottant)

Polices

- **Bricolage Grotesque** — display (titres, marques)
- **Inter** — texte courant
- **JetBrains Mono** — données techniques, scores, timestamps

Structure des composants

```
src/
├─ App.tsx           # Routing + auth init + section composition
├─ main.tsx          # Entry point
├─ components/
│   ├─ Navigation.tsx    # Sticky header + hamburger drawer mobile
│   ├─ StickyCountdown.tsx # Pill countdown précis à la seconde
│   ├─ Hero.tsx          # Title + lede + countdown grid + Lottie ball
│   ├─ Groups.tsx        # 12 groupes × 4 équipes + continental champions
│   ├─ Schedule.tsx      # Match list par date, timezone auto-détectée
│   ├─ Bracket.tsx       # KO tree R32 → Final (visuel)
│   ├─ Stadiums.tsx      # 16 stades par pays hôte
│   ├─ Players.tsx       # 23 joueurs starters + rating maison
│   ├─ Predictions.tsx   # Scores picker + share link
│   ├─ Leaderboard.tsx   # Top 50 utilisateurs (points / accuracy)
│   ├─ DailyMatches.tsx  # Aggregator 17 compétitions du jour
│   ├─ BracketWizard.tsx # Wizard 8 steps groupes → final + export PNG
│   ├─ PublicProfile.tsx # Page /u/:slug d'un bracket publié
│   ├─ AuthModal.tsx     # Sign up / log in / Google / magic link
│   ├─ UserMenu.tsx      # Avatar + tier + KPI dans la nav
│   ├─ AtlasLions.tsx    # Easter egg layer (toast + Konami)
│   ├─ Footer.tsx        # Marque + sources + dates
│   └─ SectionHeader.tsx # Eyebrow + title + sub (DRY)
├─ data/
│   ├─ teams.ts          # 48 équipes + drapeaux + confédération
│   ├─ matches.ts        # Fixtures opening + KO IDs
│   ├─ stadiums.ts       # 16 venues + lat/lng/capacité
│   ├─ players.ts        # 23 starters + algorithmme rating
│   └─ clubLogos.ts      # CDN API-Football pour les logos clubs
├─ store/
│   ├─ auth.ts           # Zustand auth + Supabase wrapper
│   ├─ predictions.ts    # Picks + share encoding + cloud sync
│   ├─ bracket.ts        # Full bracket state + publish/load
│   └─ (flagCycle.ts retiré pour cause de lag)
└─ lib/
```

```

└─ supabase.ts      # Client + SUPABASE_CONFIGURED flag
└─ api.ts           # Fetch client Worker + types ESPN
└─ utils.ts         # cn, timeUntil, fmtDate, userTimezone
└─ morocco.ts       # Quotes + Konami code + odds humour

```

5. Worker Cloudflare (wc26-api)

Endpoints exposés

Méthode	Path	TTL cache	Description
GET	/health	—	Ping de service
GET	/scoreboard	30s	Live scores WC26
GET	/fixtures	1h	Schedule complet WC26
GET	/standings	1h	Classements de groupe
GET	/match/:id	30s	Résumé d'un match
GET	/teams/:code	24h	Roster d'une équipe
GET	/today?date=YYYYMMDD	30s live / 5min sinon	Aggregateur multi-compétitions

Compétitions agrégées par /today

Slug ESPN	Label	Tier
fifa.world	FIFA World Cup 26	0
uefa.champions	UEFA Champions League	1
uefa.europa	UEFA Europa League	1
uefa.europa.conf	UEFA Conference League	1
eng.1	Premier League	2
esp.1	LaLiga	2
ita.1	Serie A	2
ger.1	Bundesliga	2
fra.1	Ligue 1	2
por.1	Liga Portugal	3
ned.1	Eredivisie	3
tur.1	Süper Lig	3
mex.1	Liga MX	3
usa.1	MLS	3

sau.1	Saudi Pro League	3
arg.1	Liga Profesional	3
fifa.friendly	International Friendlies	4

Cron */5 * * * *

Toutes les 5 minutes, le Worker :

1. Pull /scoreboard ESPN sur WC26
2. Filtre les matchs avec `status.type.state === "post"` (terminés)
3. Mappe l'ID ESPN → ID interne `M01..M104` (stub pour v1, à compléter post-tirage officiel)
4. UPSERT dans `match_results` Supabase via la service role key
5. Le **trigger SQL `score_predictions_on_result`** se déclenche automatiquement et calcule les points pour tous les utilisateurs qui ont prédit ce match

Variables et secrets

- `SUPABASE_URL` (var publique)
- `SUPABASE_SERVICE_KEY` (secret, set via `wrangler secret put`)
- KV namespace `CACHE` (binding `env.CACHE`)

Déploiement

```
cd worker
npx wrangler deploy
```

6. Schéma de base Supabase

Tables

profiles

Stocke l'identité d'un utilisateur et son agrégat de scoring.

Colonne	Type	Description
id	UUID PK (FK auth.users)	Lié à Supabase Auth
alias	text UNIQUE	Pseudo affiché
country	text	Pays optionnel
avatar_url	text	URL avatar
total_points	int	Somme des points
total_predictions	int	Total des picks faits
resolved_predictions	int	Picks dont le résultat est connu

accuracy_pct	numeric(5,2)	% précision (total_points / (resolved × 100) × 100)
current_streak	int	Streak en cours (consécutifs ≥ 30 pts)
best_streak	int	Meilleur streak historique
tier	text	Rookie / Amateur / Pro / Elite / Legend
created_at, updated_at	timestampz	Timestamps

predictions

Une ligne par (utilisateur, match) avec le pronostic.

Colonne	Type	Description
id	bigserial PK	
user_id	UUID FK	
match_id	text	M01..M104, R32-1, FINAL
home_score, away_score	int	Pick du score
scorer_ids, card_player_ids	text[]	Pour v2 (buteurs, cartons)
points	int (nullable)	Calculé après résultat
points_breakdown	jsonb	{exact:100} ou {winner:30} etc.

match_results

Résultats officiels publiés par le cron Worker.

Colonne	Type	Description
match_id	text PK	
home_score, away_score	int	Score final
scorer_ids, card_player_ids	text[]	
finished_at	timestampz	

bracket_predictions

Une ligne par utilisateur, la prédiction de tout le tournoi.

Colonne	Type	Description
id	bigserial PK	
user_id	UUID FK UNIQUE	Un seul bracket par user

group_standings	jsonb	{ "A": ["MAR", "MEX", "AUS", "CAN"], ... }
third_place_advancing	text[]	8 codes équipes
ko_winners	jsonb	{ "R32-1": "MAR", ..., "FINAL-1": "MAR" }
third_place_winner, final_winner	text	Codes
is_published, share_slug	boolean / text	Partage public
total_points, group_points, ko_points, final_points	int	À computer en Phase 3

Vue publique `leaderboard`

```
SELECT id, alias, country, avatar_url,
       total_points, resolved_predictions, accuracy_pct,
       current_streak, best_streak, tier
FROM profiles
WHERE resolved_predictions > 0
ORDER BY total_points DESC, accuracy_pct DESC
LIMIT 100;
```

Accessible en lecture par tout le monde (auth `anon` et `authenticated`).

Vue publique `public_brackets`

Brackets `is_published = true` joints avec l'alias et le tier du profil.

Trigger de scoring

```
CREATE OR REPLACE FUNCTION compute_points(p_home, p_away, r_home, r_away)
RETURNS TABLE (points int, breakdown jsonb)
```

Logique :

- **Score exact** → 100 pts
- **Vainqueur + écart correct** → 60 pts
- **Vainqueur seul** → 30 pts
- **Nombre total de buts correct** → 20 pts
- **Sinon** → 0 pt

À l'insert/update sur `match_results` , le trigger `score_predictions_on_result` :

1. Boucle sur toutes les `predictions` pour ce match dont `points IS NULL`
2. Calcule les points via `compute_points()`
3. Met à jour `predictions.points` et `predictions.points_breakdown`
4. Met à jour `profiles` (`total_points`, `accuracy_pct`, `current_streak`, `best_streak`, `tier`)

Politiques RLS

- `profiles` : lecture publique (leaderboard), écriture par le owner
 - `predictions` : lecture/écriture/update par le owner uniquement
 - `match_results` : lecture publique, écriture réservée au service role
 - `bracket_predictions` : owner CRUD + lecture publique si `is_published`
-

7. Authentification

Trois méthodes de login, toutes via Supabase Auth :

1. Email + mot de passe

- Signup → email de confirmation envoyé via Resend
- Email contient un lien magique (template HTML brandé WC26 Live, fond `#0a0a0f` , dégradé gold)
- Une fois confirmé, login illimité avec password
- Reset password disponible via "Forgot password?"

2. Magic Link (one-time)

- Pour les utilisateurs qui ne veulent pas créer de mot de passe
- Click "Use magic link instead" dans le modal
- Email reçu → clic → connecté

3. Google OAuth

- Bouton "Continue with Google" en haut du modal
- OAuth client créé dans Google Cloud Console (`786367888791-...`)
- Consent screen publié en Production → tout utilisateur Google peut se connecter
- Callback URL : `https://ssvvojhxotlbcdosiog.supabase.co/auth/v1/callback`
- Origins autorisées : `localhost:5173` , `wc26.mehdijabry.dev`

Sessions

- Tokens JWT stockés dans `localStorage` via Supabase JS
- `autoRefreshToken: true` → renouvellement transparent
- Logout via `supabase.auth.signOut()` (efface tout côté client)

Profil

Création automatique d'un profil au signup via le trigger SQL `handle_new_user` qui insert une ligne dans `profiles` avec un alias par défaut `fan_XXXXXX` .

8. Système de pronostics

Pronostics simples (Predictions section)

- L'utilisateur pick un score pour chaque match de la phase de groupe
- State local persistant via Zustand + `localStorage` (clé `wc2026-predictions`)
- Si l'utilisateur est connecté, chaque pick est immédiatement `upsert` dans `predictions` Supabase
- Sinon, les picks restent en local et sont sync au prochain login
- Lien de partage généré via `btoa(JSON.stringify(picks))` → URL `?picks=...`

Bracket Predictor (8-step wizard)

Étape	Action
1. Group standings	Rank 1er / 2e / 3e / 4e par groupe avec arrows ▲▼
2. Best 3rd-placed	Pick 8 sur les 12 troisièmes pour qualifier en R32
3. Round of 32	Click winners (matchups générés depuis steps 1+2)
4. Round of 16	Click winners (depuis step 3)
5. Quarter-finals	Click winners (depuis step 4)
6. Semi-finals	Click winners (depuis step 5)
7. 3rd place + Final	Click 3e place + Final winner
8. Publish & export	Save / Download PNG / Publish

Export PNG du bracket

- Composant React `<BracketPoster />` rend un layout complet (12 groupes + 5 colonnes KO + champion + 3e place)
- `html-to-image` capture le DOM et produit un PNG **2x retina-ready** (`pixelRatio: 2`)
- Téléchargement déclenché via `<a download>`
- Fichier nommé `wc26-bracket-{alias}.png`

Publish & partage

- `bracket_predictions.is_published = true` et `share_slug = alias`
- URL publique : `/u/{alias}`
- La page `<PublicProfile />` charge le bracket depuis la vue `public_brackets` (RLS l'autorise même pour `anon`)
- Layout dédié (header simplifié + champion + 3e + groupes + colonnes KO)

9. Scoring

Formule (par match)

Voir aussi section "Schéma de base" → trigger `compute_points` .

- Score exact : 100 pts
- Vainqueur + écart : 60 pts
- Vainqueur uniquement : 30 pts
- Total buts correct (mais perdu sur le vainqueur) : 20 pts
- Sinon : 0 pt

Tiers

Tier	Seuil
Rookie	< 500 pts
Amateur	≥ 500 pts
Pro	≥ 2 000 pts

Elite	$\geq 5\,000$ pts
Legend	$\geq 10\,000$ pts

Streak

- Augmente de 1 quand un pick rapporte ≥ 30 pts
- Reset à 0 quand un pick rapporte 0 pt
- `best_streak` conserve le record historique

Leaderboard

- Vue publique `leaderboard` (top 100)
- Affichée avec deux tabs : **Total points** et **Accuracy %**
- Highlight si l'utilisateur connecté est dans le top 100
- Mise à jour automatique à chaque trigger de scoring

10. Sources de données

ESPN public API

- Base URL : `https://site.api.espn.com/apis/site/v2/sports/soccer`
- Pas de clé requise, pas de rate limit documenté (mais on cache pour éviter le hammering)
- Format JSON propre, type `EspnEvent` reflète leur schéma
- Utilisé pour : scores live, fixtures, classements, rosters

API-Football media CDN

- `https://media.api-sports.io/football/teams/{id}.png`
- Logos haute résolution des clubs
- Mapping interne dans `src/data/clubLogos.ts`

flagcdn.com

- `https://flagcdn.com/w1280/{country_code}.png`
- Drapeaux haute résolution
- Utilisé pour les easter eggs (initialement pour l'animation des titres, retiré pour cause de lag)

Données statiques (`src/data/`)

- `teams.ts` : 48 équipes WC26, drapeaux emoji, confédération, ranking FIFA (snapshot juin 2026)
- `matches.ts` : fixtures officielles annoncées par FIFA
- `stadiums.ts` : 16 venues hôtes avec capacité, ville, pays
- `players.ts` : 23 stars + algorithme custom de rating

Algorithme de rating joueur (WC26 Live Score)

Composite 0-100 :

- **Consistency** (15) : $\min(\text{matches}/30, 1) \times 15$
- **Output** position-weighted (30) : poids différents pour ATT/MID/DEF/GK
 - ATT : goals $\times 2$ + assists
 - MID : goals + assists $\times 1.2$ + keyPasses $\times 0.5$
 - DEF : interceptions $\times 0.4$ + tackles $\times 0.3$ - reds $\times 5$
 - GK : (saves - cleansheets) $\times 0.5$ - reds $\times 5$

- **Quality** (15) : `avgRating` ESPN × 1.5
- **Form** (20) : moyenne des 5 derniers matches × 2.5
- **Reliability** (15) : `starts/matches` × 15
- **Availability** (±) : -10 si suspendu/blessé

Tier basé sur le score :

- 85+ : Top form 🔥
- 75+ : Reliable ✓
- 65+ : Watch ↗
- < 65 : Bench

11. SEO

Tags HTML

- `<title>` : "WC26 Live · Pressing 90' — World Cup 2026 scores, brackets & predictions"
- `<meta name="description">` : 160 chars EN + mention "coupe du monde" pour FR
- `<meta name="keywords">` : World Cup, WC26, FIFA, football, Atlas Lions, Pressing 90, news foot, ...
- `<link rel="canonical">` : `https://wc26.mehdijabry.dev/`
- Open Graph + Twitter Card avec image OG (à venir)
- `<meta name="robots" content="index, follow, max-image-preview:large">`
- `<meta http-equiv="content-language" content="en, fr">`

Données structurées JSON-LD

Trois objets dans `@graph` :

1. **SportsEvent** — FIFA World Cup 2026, dates, sport, lieux (US/MX/CA), URL, statut
2. **Organization** — Pressing 90', logo, sameAs mehdijabry.dev
3. **WebSite** — avec `SearchAction` (sitelink search box dans les SERPs Google)

Sitemap

`public/sitemap.xml` avec 9 URLs :

- `/` (priority 1.0, changefreq hourly)
- `/#groups` , `/#schedule` , `/#bracket` , `/#stadiums` , `/#predict` , `/#bracket-predict` , `/#today` , `/#leaderboard`

robots.txt

```
User-agent: *
Allow: /
Crawl-delay: 1
Sitemap: https://wc26.mehdijabry.dev/sitemap.xml
```

Search Console

- Propriété ajoutée : `https://wc26.mehdijabry.dev/`
- Vérification par balise HTML meta (token Google)
- Sitemap soumis et accepté (9 URLs)
- Première exploration : 7 juin 2026, 07:21 par Googlebot smartphone (mobile-first indexing)

- Statut : **"Cette URL est sur Google"**, indexation prioritaire demandée
-

12. Analytics

Cloudflare Web Analytics

- Beacon installé : `<script defer src="https://static.cloudflareinsights.com/beacon.min.js" data-cf-beacon='{"token":"d6a44cf6994e1798c8b6139f3813dd"}'></script>`
- Sans cookies, GDPR-friendly
- Mesure : pageviews, visites uniques, top pages, sources de trafic (referrers), pays, devices, navigateurs
- Temps réel (~30s de latence)
- Dashboard : `dash.cloudflare.com` → Web Analytics → `wc26.mehdijabry.dev`
- App iOS officielle Cloudflare disponible

Google Search Console

- Mesure : impressions et clics depuis Google Search uniquement
 - Mots-clés qui amènent du trafic
 - Position moyenne et CTR
 - Erreurs d'indexation et de couverture
 - Latence : 24-48h
-

13. Déploiement

Frontend (Cloudflare Pages)

```
npm run build          # build Vite → dist/
npx wrangler pages deploy dist \
  --project-name=wc26-live \
  --branch=main
```

CI/CD : le projet est aussi connecté à `github.com/mehdijabry/wc26-live`, donc tout push sur `main` redéploie automatiquement via Cloudflare Pages.

Worker (Cloudflare Workers)

```
cd worker
npx wrangler deploy
```

Domaine custom

DNS sur Porkbun :

Type	Host	Answer	TTL
CNAME	wc26	wc26-live.pages.dev	600

Cloudflare Pages provision automatiquement le certificat HTTPS (Let's Encrypt).

Variables d'environnement

`.env.local` (non versionné) :

```
VITE_SUPABASE_URL=https://ssvvojhxyotlbcdosiog.supabase.co
VITE_SUPABASE_ANON_KEY=sb_publishable_...
```

Ces vars sont aussi à configurer dans Cloudflare Pages → Settings → Environment variables si on passe au build via CF (au lieu de wrangler CLI local).

14. Easter eggs et signature éditoriale

Pressing 90'

- Marque parente, affichée en sous-titre de WC26 Live partout (nav + footer + drawer mobile + PublicProfile)
- "Pressing" en blanc, "90'" en accent-red
- Référence tactique foot (gegenpressing) + double sens média (the press)

Atlas Lions (🇲🇦)

- **Konami code** (↑↑↓↓←→←→BA) → overlay confettis rouge/jaune/vert + "Atlas Lions World Champions 2026"
- **Toast occasionnel** bottom-center toutes les ~90s avec une quote rotative parmi 12 citations marocaines
- **Console intro** : ASCII rouge/blanc/vert au load, plus une commande `window.morocco()` qui retourne une quote random
- **Groupe A** : highlight de la card Morocco avec gradient subtil
- **Player cards** : 5 lions (Hakimi, Bono, Ziyech, En-Nesyri, Ounahi) avec stripe gauche tricolore subtile
- **Predictions section** : tongue-in-cheek odds bar (MAR 99.8 %, others 0.05 % chacun)

Continental champions

Un titre subtil par confédération, affiché dans les groupes :

- CAF champ → MAR
 - UEFA champ → ESP
 - CONMEBOL champ → ARG
 - AFC champ → JPN
 - CONCACAF champ → USA
 - OFC champ → NZL
-

15. Roadmap

Phase courante (✅ DONE)

- Frontend MVP complet (10 sections + bracket wizard + public profile)
- Worker proxy ESPN + cron 5 min
- Supabase auth (password + Google OAuth + magic link) + Resend SMTP brandé
- SEO + analytics + indexation
- Custom domain HTTPS

Phase 3 (à venir)

- Mapping officiel des ID ESPN → ID interne M01..M104 (post-tirage)
- Wiring Schedule + Bracket avec données ESPN live (au lieu du mock)
- Predictions multi-niveau : buteurs, cartons, lineups (5 pts par titulaire correct)
- Server-side scoring du bracket complet (groupes + KO + final)
- Notifications email aux utilisateurs quand un match prédit a un résultat

Phase 4 (futur)

- Application iOS (PWA + push notifs)
- Monétisation : Google AdSense + sponsor placement
- Acquisition payante : X Ads, Reddit Ads, Google Ads sur mots-clés foot
- Multilingue : FR + ES + AR (arabe pour audience MENA)

16. Annexes

Commandes utiles

```
# Dev local
npm run dev                                # Vite dev server :5173

# Build production
npm run build                              # → dist/

# Deploy frontend
./worker/node_modules/.bin/wrangler pages deploy dist \
  --project-name=wc26-live \
  --branch=main \
  --commit-dirty=true

# Deploy Worker
cd worker && npx wrangler deploy

# Logs Worker en live
cd worker && npx wrangler tail

# Supabase logs
# → dashboard supabase.com/dashboard → Logs

# DNS check
dig +short CNAME wc26.mehdijabry.dev @8.8.8.8

# Vérifier les meta SEO
curl -s https://wc26.mehdijabry.dev/ | grep -E "title|google-site-
verification|cloudflareinsights"
```

Structure du repo

```
wc26-live/
├── .env.example
├── .gitignore
```

```

├── README.md
├── docs/                                # ← cette documentation
│   ├── TECHNICAL.md
│   └── OVERVIEW.md
├── index.html
├── package.json
├── tailwind.config.js
├── tsconfig.json
├── vite.config.ts
├── public/                             # assets statiques
│   ├── wc26-emblem.svg
│   ├── favicon.svg
│   ├── icons.svg
│   ├── robots.txt
│   └── sitemap.xml
├── src/                                # frontend React
│   ├── App.tsx
│   ├── main.tsx
│   ├── index.css
│   ├── vite-env.d.ts
│   ├── components/
│   ├── data/
│   ├── store/
│   └── lib/
├── supabase/
│   └── migrations/
│       ├── 001_init.sql                # profiles + predictions + scoring
│       └── 002_bracket.sql             # bracket_predictions + public_brackets
└── worker/                             # Cloudflare Worker (sous-projet)
    ├── package.json
    ├── tsconfig.json
    ├── wrangler.toml
    └── src/
        └── index.ts

```

Contact & maintenance

- **Auteur** : Mehdi Jabry — mehdijabry.dev
- **Repo** : github.com/mehdijabry/wc26-live
- **Production** : wc26.mehdijabry.dev
- **Pour signaler un bug** : ouvrir une issue sur le repo GitHub
- **Pour proposer une feature** : pull request ou issue avec label `enhancement`

WC26 Live n'est pas affilié à la FIFA. Données issues d'APIs publiques (ESPN, FIFA.com, Sofascore).